PCS II — Exam Simulation 1 (Phase 1 — paper and pencil)

General instructions

- Write your answers **on paper**.
- In **Phase 2** you will implement in Python, in the Colab notebook, **the same algorithms** you design here for Exercises 2 and 3 (Exercise 1 is paper-only). Keep this in mind: an algorithm you cannot implement is of little use.
- Time complexities must be expressed with the **Big-O notation**, with a short justification (count how many times the main operations are executed).

Exercise 1 — Counting matching nucleotide pairs (runtime complexity)

You are given a DNA string `s` of length `n` over the alphabet `{A, C, G, T}`. Positions are numbered from `0` to `n-1`.

Two positions `i < j` form a **matching pair** if `s[i] = s[j]`, i.e. the two positions contain the same nucleotide.

Below are two algorithms that both compute the number of matching pairs of `s`.

Algorithm A

```
MATCHINGPAIRS-A(s)
  n <- length(s)
  count <- 0
  for i <- 0 to n-2
    for j <- i+1 to n-1
      if s[i] = s[j] then
        count <- count + 1
  return count
```

Algorithm B

```
MATCHINGPAIRS-B(s)
  n <- length(s)
  cA <- 0;  cC <- 0;  cG <- 0;  cT <- 0
  for i <- 0 to n-1
    if s[i] = 'A' then cA <- cA + 1
    else if s[i] = 'C' then cC <- cC + 1
    else if s[i] = 'G' then cG <- cG + 1
    else cT <- cT + 1
  return cA*(cA-1)/2 + cC*(cC-1)/2 + cG*(cG-1)/2 + cT*(cT-1)/2
```

Example

Input

s = “GATTACA”

Expected output

4

Explanation. `A` appears at positions 1, 4, 6, giving the pairs (1,4), (1,6), (4,6); `T` appears at positions 2, 3, giving the pair (2,3). `G` and `C` appear only once. In total 3 + 1 = 4 matching pairs.

What to deliver

1. For the input `s = "GATTACA"` (`n = 7`), count **exactly** how many times the comparison `s[i] = s[j]` is executed by Algorithm A. Then give the exact number of executions **as a function of `n`** for a generic input of length `n`.
2. The **time complexity** of Algorithm A in Big-O notation, with a justification based on point 1.
3. The **time complexity** of Algorithm B in Big-O notation, with a justification (how many times is the body of the loop executed? how much work is done after the loop?).
4. Explain in a few lines **why Algorithm B returns the same result** as Algorithm A. *Hint:* how many pairs of positions can be formed from `c` positions that all contain the same nucleotide?

Exercise 2 — Cell division (recursion)

A biologist starts a culture with `p` cells. At every **generation**, every cell in the culture divides into two cells, so the number of cells doubles.

The number of cells after `n` generations, starting from `p` cells, can be defined as follows:

- after `0` generations there are `p` cells (base case);
- after `n >= 1` generations there are **twice** as many cells as after `n - 1` generations (recursive case).

Task

Write pseudocode for a **recursive** algorithm `CELLS(p, n)` that returns the number of cells after `n` generations, starting from `p` cells.

You may assume that `p >= 1` and `n >= 0` are integers.

Solutions that use a loop (`for` / `while`), or that directly compute a power (`2^n`), will **not** be accepted: the goal of the exercise is the recursion.

Example

Input

p = 1, n = 3

Output

8

(1 cell → 2 cells → 4 cells → 8 cells.)

Input

p = 5, n = 2

Output

20

What to deliver

1. The pseudocode of `CELLS(p, n)`, clearly identifying the **base case** and the **recursive case**.
2. A trace of your algorithm on the input `p = 1, n = 3`, showing the sequence of recursive calls and the value returned by each of them.
3. The **number of recursive calls** performed for a given `n`, and the resulting **time complexity** in Big-O notation (assume that each call performs a constant amount of work in addition to the recursive call).

Exercise 3 — The most homogeneous group of samples

A laboratory has measured a quantity (for instance the expression level of a gene) in `n` biological samples. The measurements are stored in a list `values` of `n` integers (positive, negative or zero), one per sample.

For a control experiment, the researchers must select **exactly `k` samples** (with `2 <= k <= n`) that are **as similar as possible** to each other. The **spread** of a group of samples is defined as

spread = (largest value in the group) − (smallest value in the group).

Your task is to compute the **minimum possible spread** of a group of exactly `k` samples.

Input format

- A list `values` of `n` integers.
- An integer `k`, with `2 <= k <= n`.

Output

A single integer: the minimum spread achievable by selecting exactly `k` samples.

Example

Input

values = [10, 100, 300, 200, 1000, 20, 30]

k = 3

Expected output

20

Explanation. Selecting the samples with values 10, 20 and 30 gives spread 30 − 10 = 20. No other group of 3 samples has a smaller spread (for instance, {100, 200, 300} has spread 200).

Second example

values = [1, 4, 7, 2]

k = 2

Expected output

1

(The group {1, 2} has spread 1. Note that for `k = 2` the problem is exactly the "Minimum Absolute Difference" problem seen in the tutorial.)

Third example

values = [100, 200, 300, 350, 400, 401, 402]

k = 3

Expected output

2

Important

A **brute-force** algorithm that examines **every possible group** of `k` samples and keeps the one with the smallest spread is **not acceptable**: the number of such groups grows extremely fast with `n` (for `n = 100` and `k = 10` there are more than 10^13 groups). You must propose an algorithm that is **asymptotically more efficient** than brute force and that works for `n` in the order of millions.

What to deliver

1. **Pseudocode** of an algorithm that computes the minimum spread.
2. The **algorithmic technique** your algorithm is based on, a description of the **strategy** and a **justification of its correctness**: why is your algorithm guaranteed to find the minimum spread without examining all possible groups?
3. The **time complexity** and **space complexity** of your algorithm in Big-O notation, with a short justification, and a comparison with the brute-force approach.